

Claude Code 도구 가이드

AI가 만든 웹앱을 스스로 테스트(gstack)하고, 작업을 반복(loop)하며, 목표를 정하고(goal), 자주 쓰는 작업을 묶고(skills), 외부 도구·기억을 연결하는(MCP) — 우리 워크스페이스의 핵심 도구 5종.

gstack — 헤드리스 브라우저 QA

loop — 반복 실행

goal — 세션 목표

skills — 슬래시 스킬

MCP — 외부 도구·기억 연결

01 이 문서는

다섯 도구를 "처음 듣는 사람"도 바로 감 잡도록, 개념 → 왜 필요한가 → 실제 명령 → 우리 프로젝트 적용 순으로 정리했습니다.

gstack

AI가 브라우저를 운전해 우리가 만든 화면을 클릭·입력·확인까지 대신 검증하는 QA 도구.

loop

같은 작업을 일정 간격으로 반복 실행. 배포 감시·주기 점검 같은 반복 작업을 자동화.

goal

작업이 향할 목표·방향을 미리 못 박기. AI가 긴 작업 내내 그 목표 기준으로 판단·종료.

skills

자주 하는 작업 절차를 슬래시 명령 하나로. `/team-init`, `/prototype-scaffold` 처럼 한 줄로 호출.

MCP

외부 도구·데이터·장기기억을 표준 방식으로 연결. 우리는 세션 간 기억용 `memory` 서버를 쓴다.

한 줄 요약 — gstack은 "만든 걸 검증", loop는 "그 검증을 반복", goal은 "무엇을 향해·언제까지", skills는 "반복 작업을 한 줄로", MCP는 "AI에 외부 능력·기억을 끼우기"입니다.

02 gstack — AI가 운전하는 헤드리스 브라우저

화면 없는(headless) 크로미움을 띄워두고, 명령 한 줄씩으로 페이지를 조작·검사합니다. 첫 호출 때 자동 시작(~3초), 이후 명령은 ~0.1~0.2초.

무엇을 하나

- 페이지 이동·읽기 — 주소로 이동하고 본문 텍스트·콘솔 오류·네트워크 요청을 확인
- 요소 조작 — 버튼 클릭, 입력칸 채우기, 파일 업로드, 드롭다운 선택
- 상태 검증 — "특정 요소가 보이는가 / 활성화됐는가 / 텍스트가 맞는가" 단언(assert)
- 증거 남기기 — 스크린샷, 주석(annotated) 스크린샷, 변경 전후 diff
- 반응형 점검 — 모바일/태블릿/데스크톱 화면을 한 번에 캡처

왜 필요한가 — 우리는 Gradio·웹 데모를 빠르게 찍어냅니다. 만들 때마다 "버튼이 눌리나? 이미지가 뜨나? 콘솔 에러는 없나?"를 손으로 확인하면 느립니다. gstack은 그 클릭·확인을 AI가 대신하고 스크린샷으로 증거까지 남깁니다.

핵심 사용 흐름 — 스냅샷 → 조작 → diff

```
# 1) 페이지 열기
$B goto http://127.0.0.1:7860

# 2) 조작 가능한 요소 목록 보기 (@e1, @e2 ... 참조 번호가 붙음)
$B snapshot -i

# 3) 참조 번호로 입력 · 클릭
$B fill @e3 "안녕? 한 문장으로 자기소개 해줘."
$B click @e5

# 4) 무엇이 바뀌었는지 확인하고 증거 남기기
$B snapshot -D          # 직전 대비 변경점(diff)
$B is_visible ".message" # 응답이 떴는지 단언
$B console              # JS 에러 없는지
$B screenshot /tmp/after.png
```

자주 쓰는 명령	설명
<code>goto <url></code>	페이지 이동
<code>snapshot -i</code>	클릭·입력 가능한 요소를 번호(@e)와 함께 표시
<code>click / fill / upload / select</code>	요소 조작

<code>snapshot -D</code>	직전 상태 대비 변경점 diff
<code>is visible/enabled/checked <sel></code>	상태 단언(테스트 통과 판정)
<code>screenshot / responsive</code>	스크린샷 · 반응형 3종 캡처
<code>console / network</code>	JS 오류 · 실패한 요청 확인

03 loop — 작업을 주기적으로 반복

프롬프트나 슬래시 명령을 일정 간격으로 다시 실행시킵니다. 한 번 시키고 잊는 게 아니라, "계속 지켜보거나 반복해야 하는 일"에 씁니다.

무엇을 하나

- 주기 실행 — "5분마다 배포 상태 확인" 같은 고정 간격 반복
- self-paced(자율 페이스) — 간격을 안 주면 AI가 스스로 "언제 다시 돌릴지" 판단해 반복
- 상태 폴링 — CI/배포/외부 작업이 끝났는지 반복 확인

사용법

```
# 5분마다 QA 점검 반복
/loop 5m /qa
```

```
# 간격 생략 → AI가 알아서 페이스 조절 (self-paced)
/loop 배포가 끝났는지 확인하고 끝나면 알려줘
```

loop의 "끝(종료조건)"

loop는 "언제 멈출지(종료조건)"가 명확할수록 좋습니다. 그 종료조건이 바로 다음 장의 **goal**입니다. gstack의 `is visible` 같은 단언이 그대로 loop의 합격·종료 판정이 됩니다.

04 goal — 작업이 향할 목표 정하기

Claude Code에서 한 작업/세션 전체가 향하는 목표·방향을 미리 정해두는 기능입니다. 목표를 처음에 명확히 한 번 주면, AI가 긴 작업 내내 그 목표를 기준으로 판단하고, 결과지로 새도 목표로 되 돌아옵니다.

두 가지 "goal"이 있어요

- **Claude Code** `/goal` — 작업 전체의 방향. "이 세션은 무엇을 이루려는가."
- **loop의 goal** — 반복의 종료조건. "이 반복을 언제 멈출까."

둘 다 "정해진 목표를 향해 일하게" 만드는 장치이고, 범위만 다릅니다.

왜 효과적인가

길고 자율적인 작업일수록, 목표를 처음에 한 번에 구체적으로 주면 AI가 더 정확하고 효율적으로 끝까지 갑니다. 중간에 되묻거나 엉뚱한 길로 새는 일이 줄어듭니다.

사용법

```
# 세션 목표를 한 줄로 못 박기
```

```
/goal 포트홀 데모를 완성한다: 이미지 업로드 시 손상 영역을 표시하고  
신뢰도(%)를 출력하면 완료. Gradio 한 파일로, MOCK 먼저.
```

좋은 목표 vs 나쁜 목표 — 좋음: "배포 URL이 200을 반환하고 첫 화면에 로그인 버튼이 보이면 완료" (판정 가능). **나쁨:** "잘 되게 해줘" (합격 기준이 없어 끝을 모름).

핵심은 결과물과 합격 기준을 구체적으로 적는 것. 그러면 goal이 그대로 "완료 판정"이 되고, loop-gstack의 단언과 맞물립니다.

05 skills — 반복 작업을 슬래시 명령 하나로

스킬(skill)은 "자주 하는 작업 절차"를 미리 정의해 둔 묶음입니다. `/이름` 한 줄로 호출하면 AI가 그 절차대로 일합니다. 매번 길게 설명할 필요 없이, "표준 작업"을 버튼처럼 누르는 셈입니다.

왜 필요한가 — 5팀이 똑같은 일(팀 세팅, 프로토타입 뼈대, 기획서 양식)을 제각각 하면 결과가 들쭉날쭉합니다. 스킬은 그 절차를 **한 번 정의해 모두가 같은 방식으로 쓰게** 만들어, 산출물 품질을 일정하게 맞춥니다.

우리 프로젝트 전용 스킬

스킬	무엇을 하나	언제
<code>/team-init <팀명></code>	<code>teams/<팀명>/</code> 아래 표준 구조(팀 CLAUDE.md·기획 노트·프로토타입 폴더·README) 생성	새 팀 시작할 때
<code>/prototype-scaffold</code>	<code>prototypes/<feature>/</code> 에 Gradio/FastAPI 단일 파일 데모 스케폴드 생성(시연 시나리오 한 줄 + MOCK 함수부터)	새 기능 데모 시작할 때
<code>/planning-report</code>	지엔소프트 제출 양식 기반 한국어 기획 보고서/기능 정의서/발표자료 초안을 <code>docs/</code> 에 생성	기획 문서 쓸 때

그 외 자주 쓰는 내장·확장 스킬

- 점검·디버그 — `/qa` 동작 점검 · `/investigate` 버그 추적 · `/run` 앱 실행·확인
- 코드·리뷰 — `/code-review` 변경 리뷰 · `/security-review` 보안 점검 · `/simplify` 정리
- 문서·맥락 — `/make-pdf` 페이지→PDF · `deep-research` 다출처 리서치

skill vs subagent vs MCP — 헷갈리지 않기

- skill** = "정해진 작업 절차"를 슬래시로 호출 (무엇을 어떤 순서로 할지).
- subagent** = 특정 역할의 전문 AI에게 위임(예: `vlm-researcher` , `rag-architect`). 누가 할지.
- MCP** = AI가 쓸 수 있는 외부 도구·데이터 연결(다음 장). 무엇으로 할지.

06 MCP — 외부 도구·데이터·기억 연결하기

MCP(Model Context Protocol)는 Claude Code가 외부 도구·데이터에 **표준 방식으로 연결되는** 규약입니다. "MCP 서버" 하나가 곧 "도구 묶음 하나"이고, 연결하면 AI가 그 도구들을 직접 호출할 수 있습니다.

왜 필요한가 — 기본 AI는 "이 대화 안의 정보"만 압니다. MCP를 붙이면 파일시스템·DB·웹·외부 API, 그리고 세션을 넘는 **장기 기억**까지 AI의 손이 닿습니다. 우리는 그중 **기억(memory)**을 먼저 도입했습니다.

지금 쓰는 서버 — memory (장기 기억)

세션이 바뀌어도 결정사항·진행상황을 이어가기 위한 서버입니다. SQLite에 저장되고, **자동 로드되지 않아 평소 토큰 소모 0** — 필요할 때만 검색해서 꺼냅니다.

도구	설명
<code>memory_save(content, tags)</code>	한 건의 사실/결정을 저장. 태그는 공백 구분.
<code>memory_search(query)</code>	키워드로 기억 검색(여러 단어는 AND).
<code>memory_recent(limit)</code>	최근 저장한 기억을 시간 역순으로.
<code>memory_delete(id)</code>	잘못된 기억 삭제.

```
# 구현: .claude/mcp/memory_server.py (Python stdlib만, 외부 패키지 0)
# 저장소: .claude/memory.db (SQLite, 커밋 금지)
# 사용 예 - 세션 시작 시 맥락 복원
memory_search("포트폴리오 데모 진행상황")
# 중요한 결정이 생기면 즉시 저장 (한 건 = 한 사실)
memory_save("api-test 데모는 Gradio 단일파일로 확정", tags="prototype 결정")
```

MCP 추가하는 법 (요약)

프로젝트 루트 `.mcp.json` 의 `mcpServers` 에 항목을 추가하고 `/mcp` 로 연결을 확인합니다. 시크릿은 코드/JSON에 하드코딩하지 말고 `.env` 로 주입합니다.

```
{
  "mcpServers": {
    "memory": { "command": "python.exe", "args": [".claude/mcp/memory_server.py"] }
  }
}
```

유지 규칙

도입한 MCP 서버 목록·용도는 `docs/mcp-servers.md` 에 정리합니다. 새 MCP를 붙일 때마다 그 문서에 항목을 추가하세요(현재 후보: filesystem · fetch · sqlite · 벡터DB · github 등).

07 다 엮기 — 우리 프로젝트 적용

`prototypes/api-test` Gradio 데모를 예로, 다섯 도구를 한 흐름으로 묶으면 이렇게 됩니다.

① skill — 뼈대부터

```
/prototype-scaffold # prototypes/api-test 뼈대 + MOCK 함수 생성
```

② goal — 목표를 먼저

```
/goal api-test 앱이 떠 있고, 메시지를 보내면 Claude 응답이  
화면에 표시되면 "정상"으로 본다.
```

③ gstack — 목표를 검증

```
$B goto http://127.0.0.1:7860  
$B is visible "textarea" # 입력창 존재 = 1차 합격  
$B snapshot -i  
$B fill @e3 "연결 테스트"  
$B click @e5  
$B is visible ".message" # 응답 표시 = goal 도달  
$B screenshot docs/qa-evidence.png
```

④ loop — 검증을 반복

```
/loop 10m /qa # 10분마다 위 점검 자동 반복, goal 도달 시 보고 후 종료
```

⑤ MCP — 진행상황을 기억

```
memory_save("api-test QA 통과, 응답 정상 표시 확인", tags="api-test qa 진행상황")
```

흐름 — skill(뼈대) → goal(목표) → gstack(검증) → loop(반복) → MCP(기억). 목표가 곧 합격·종료
기준이라 도구들이 자연스럽게 맞물리고, 진행상황은 세션이 바뀌어도 이어집니다.

08 확장 기능 — 더 멀리 가기

gstack은 단일 명령뿐 아니라 워크플로 슬래시 스킴 묶음을 제공합니다. 상황에 맞게 골라 쓰세요. (대부분 gstack 브라우저 실행이 필요 → 09~10장 차단 해제 후 사용)

분류	스킬	언제
점검·디버그	<code>/qa</code> <code>/qa-only</code> <code>/investigate</code> <code>/health</code>	"이거 동작해?", 버그 추적, 코드 건강도
코드·리뷰	<code>/review</code> <code>/codex</code> <code>/cso</code>	변경 리뷰, 2차 의견, 보안 점검
설계·기획	<code>/office-hours</code> <code>/plan-eng-review</code> <code>/design-consultation</code> <code>/design-review</code>	아이디어 검토, 구조/디자인 리뷰
배포·운영	<code>/ship</code> <code>/land-and-deploy</code> <code>/canary</code>	배포, 머지+배포, 배포 후 모니터링
문서·매락	<code>/make-pdf</code> <code>/context-save</code> <code>/context-restore</code> <code>/retro</code> <code>/learn</code>	페이지→PDF, 진행상황 저장·복원, 회고

고급 사용 패턴

- **chain** — 여러 명령을 JSON 한 방에 실행해 긴 흐름을 빠르게 (로그인→입력→캡처)
- **responsive** — 모바일/태블릿/데스크톱 3종 화면을 한 번에 캡처해 반응형 점검
- **cookie-import-browser** — 실제 브라우저 쿠키를 가져와 로그인 필요한 페이지까지 테스트
- **loop + /qa + goal** — 목표를 정하고, 주기적으로 자동 회귀 점검(regression)하는 루프 구성
- **/make-pdf** — 완성한 데모 화면이나 결과를 그대로 PDF 산출물로 (이 가이드는 브라우저 차단 중이라 Edge 방식으로 생성)

조합 예

`/goal` 로 "데모 합격 기준"을 정하고 → `/loop` 로 `/qa` 를 주기 실행 → 통과하면 `/ship` 으로 배포 → `/canary` 로 배포 후 점검. 기획부터 운영까지 한 줄씩으로 이어집니다.

09 현재 환경 구축 상태

이 PC(Windows 11) 기준으로 지금 무엇이 준비됐고 무엇이 막혀 있는지 정리.

항목	상태	비고
goal / loop / skills	바로 사용 가능	Claude Code 내장. 설치 불필요. <code>/goal</code> , <code>/loop</code> , <code>/team-init</code> 등으로 호출.
MCP <code>memory</code> 서버	연결됨	<code>.mcp.json</code> 등록, <code>/mcp</code> 로 확인. <code>stdlib</code> 만 써서 추가 설치 불필요.
gstack 설치 / 빌드	완료	<code>~/.claude/skills/gstack</code> , <code>browse</code> 바이너리 빌드됨, <code>bun 1.3.13</code> 설치됨.
gstack 브라우저 실행	차단됨	Windows Smart App Control 이 서명 안 된 <code>browse.exe</code> 실행을 막음. 아래 10 참고.
이 PDF 생성	완료	서명된 Microsoft Edge 헤드리스 인쇄로 생성(차단 영향 없음).

요점 — `goal·loop·skills·MCP(memory)`는 지금 당장 쓸 수 있습니다. `gstack`과 그 확장 스킬은 "마지막 한 칸"인 OS 보안 정책만 풀면 바로 동작합니다.

10 gstack 실행 차단 해제 (선택)

gstack의 브라우저는 외부에서 빌드된 서명 없는 실행파일이라, Windows 11의 **Smart App Control** 이 기본적으로 막습니다. 보안 기능이므로 **의미를 이해하고** 진행하세요.

주의

Smart App Control은 한 번 **끄면**, 다시 켜려면 보통 **Windows 재설정(초기화)** 이 필요합니다(되돌리기 어려움). 회사/공용 PC라면 관리자와 상의하세요.

해제 절차

- 1 **Windows 보안** 열기 (시작 → "Windows 보안" 검색)
- 2 **앱 및 브라우저 컨트롤** → **스마트 앱 컨트롤(Smart App Control)** 클릭
- 3 설정을 **끔(Off)** 으로 변경
- 4 터미널에서 동작 확인:

```
& "$env:USERPROFILE\W.claudeWskills\Wgstack\Wbrowse\Wdist\Wbrowse.exe" status
```

정상이면 서버 상태가 출력됩니다. 이후 `/qa` 등 gstack 스킬과 확장 기능을 바로 쓸 수 있습니다.

끄고 싶지 않다면

gstack 브라우저 없이도 프로젝트는 진행됩니다. goal-loop-skills-MCP는 영향 없고, 화면 확인이 필요하면 당분간 **직접 브라우저로** 데모를 열어 보면 됩니다. 자동 QA·확장 스킬은 차단 해제 후 도입해도 늦지 않습니다.

11 빠른 시작 치트시트

goal

```
/goal 완료 기준을  
구체적으로 한 줄
```

loop

```
/loop 5m /qa  
/loop 끝나면 알려줘
```

gstack

```
$B goto URL  
$B snapshot -i  
$B click @e3
```

skills (우리 전용)

```
/team-init 팀명  
/prototype-scaffold  
/planning-report
```

MCP memory

```
memory_search("키워드")  
memory_save("사실",  
tags="태그")
```

확장 슬래시 스킬

```
/qa 점검 /review 리뷰  
/ship 배포 /make-pdf
```

한 줄 흐름

```
/prototype-scaffold (뼈대) → /goal (목표) → $B gstack(검증) → /loop /qa (반복) →  
memory_save (기억). 막히면 memory_search 로 지난 맥락부터 꺼내기.
```